

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 7”

Disusun Oleh:

Zaki Misucha Andefi

2511533006

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data Pekan 7 ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini dibuat sebagai salah satu bentuk tugas praktikum pada mata kuliah Struktur Data yang bertujuan untuk memahami konsep serta implementasi algoritma pengurutan (*Sorting*) menggunakan bahasa pemrograman Java. Melalui praktikum kali ini, penulis mempelajari berbagai algoritma dasar seperti *Bubble Sort*, *Insertion Sort*, dan *Selection Sort*, serta mengimplementasikan visualisasinya menggunakan antarmuka grafis (GUI) dengan Java Swing.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi isi maupun penulisan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan pembaca mengenai implementasi algoritma *sorting* dalam pemrograman Java.

Padang, May 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Langkah Kerja.....	2
2.2 Program Iterasi Queue	2
2.3 Program Queue Array	3
2.4 Program Queue Array Driver.....	3
2.5 Program Queue Linked List.....	5
2.5 Program Reverse Data	Error! Bookmark not defined.
BAB III KESIMPULAN.....	6
DAFTAR PUSTAKA	7

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu pemrograman karena digunakan untuk mengatur, menyimpan, dan mengelola data agar proses pengolahan informasi dapat dilakukan secara efisien. Salah satu operasi paling fundamental dalam pengolahan data adalah pengurutan (*sorting*). Algoritma *sorting* berfungsi untuk menyusun sekumpulan data dalam urutan tertentu, seperti dari nilai terkecil ke terbesar (*ascending*) atau sebaliknya (*descending*). Pemilihan algoritma pengurutan yang tepat sangat memengaruhi efisiensi program, terutama ketika berhadapan dengan kumpulan data berskala besar. Beberapa algoritma pengurutan dasar yang sering dipelajari adalah *Bubble Sort*, *Insertion Sort*, dan *Selection Sort*.

1.2 Tujuan

Memahami konsep logika dari algoritma *Bubble Sort*, *Insertion Sort*, dan *Selection Sort*, serta mampu mengimplementasikan dan memvisualisasikan algoritma tersebut menggunakan antarmuka grafis (GUI) dalam bahasa pemrograman Java.

1.3 Manfaat

Meningkatkan pemahaman mengenai cara kerja algoritma pengurutan data secara *step-by-step*, melatih logika pemrograman berorientasi objek, serta memberikan pengalaman dalam membangun visualisasi proses algoritma menggunakan komponen Java GUI (Swing).

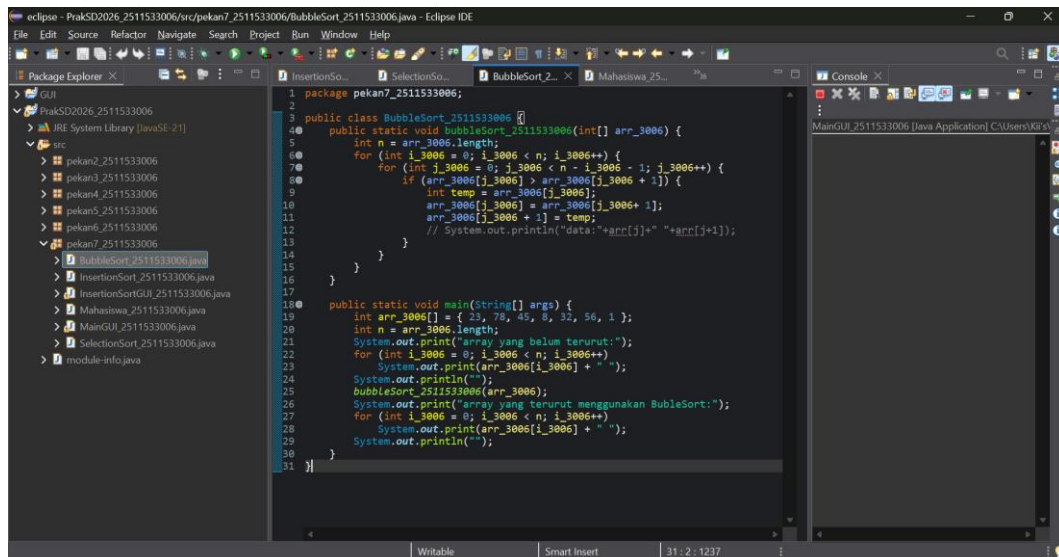
BAB II

PEMBAHASAN

2.1 Langkah Kerja

1. Membuka IDE (Eclipse).
2. Membuat package baru bernama pekan7_2511533006.
3. Menambahkan beberapa class:
 - BubbleSort_2511533006
 - InsertionSort_2511533006
 - SelectionSort_2511533006
 - InsertionSortGUI_2511533006
4. Menjalankan program satu per satu dan mengamati hasilnya.

2.2 Program Bubble Sort

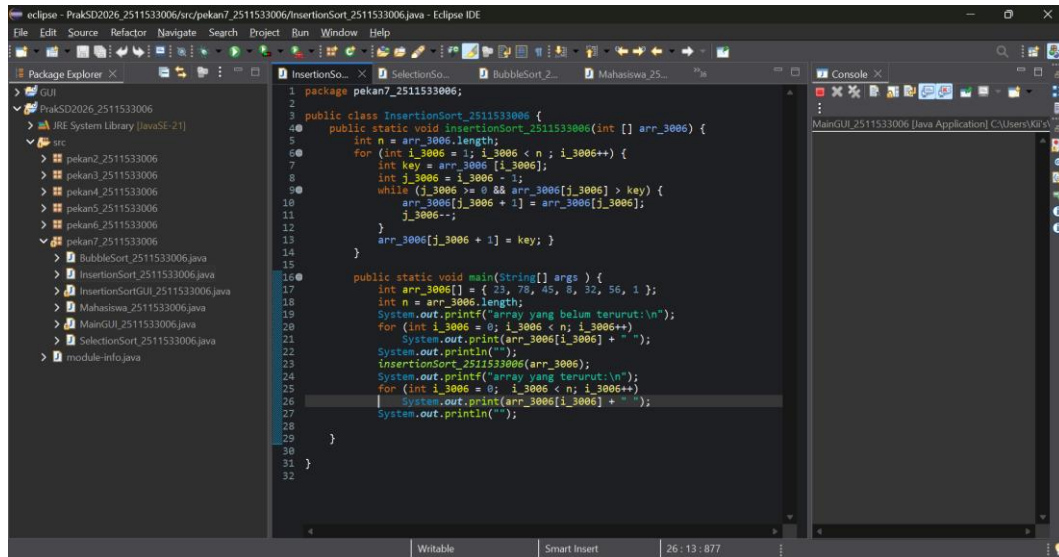


```
1 package pekan7_2511533006;
2
3 public class BubbleSort_2511533006 {
4     public static void bubbleSort_2511533006(int[] arr_3006) {
5         int n = arr_3006.length;
6         for (int i_3006 = 0; i_3006 < n; i_3006++) {
7             for (int j_3006 = 0; j_3006 < n - i_3006 - 1; j_3006++) {
8                 if (arr_3006[j_3006] > arr_3006[j_3006 + 1]) {
9                     int temp = arr_3006[j_3006];
10                    arr_3006[j_3006] = arr_3006[j_3006 + 1];
11                    arr_3006[j_3006 + 1] = temp;
12                    // System.out.println("data: "+arr[j]+ " "+arr[j+1]);
13                }
14            }
15        }
16    }
17
18    public static void main(String[] args) {
19        int arr_3006[] = { 23, 78, 45, 8, 32, 56, 1 };
20        int n = arr_3006.length;
21        System.out.println("array yang belum terurut:");
22        for (int i_3006 = 0; i_3006 < n; i_3006++) {
23            System.out.print(arr_3006[i_3006] + " ");
24        }
25        System.out.println("");
26        bubbleSort_2511533006(arr_3006);
27        System.out.println("array yang terurut menggunakan BubbleSort:");
28        for (int i_3006 = 0; i_3006 < n; i_3006++) {
29            System.out.print(arr_3006[i_3006] + " ");
30        }
31        System.out.println("");
32    }
33 }
```

Class BubbleSort_2511533006 mengimplementasikan algoritma pengurutan yang membandingkan elemen yang berdekatan dan menukarnya jika tidak dalam urutan yang benar.

Analisis: Algoritma ini menggunakan perulangan bersarang (*nested loop*). Perulangan luar berjalan sebanyak n kali, dan perulangan dalam membandingkan $arr_3006[j_3006]$ dengan $arr_3006[j_3006 + 1]$. Jika elemen sebelah kiri lebih besar, posisi keduanya ditukar menggunakan variabel sementara (*temp*). Proses ini "mengapungkan" elemen dengan nilai terbesar ke ujung kanan *array* pada setiap iterasinya. mengatur pointer *next* serta *prev* menjadi null. Class ini menjadi dasar utama dalam implementasi operasi-operasi lain pada Double Linked List.

2.3 Program Insert Sort

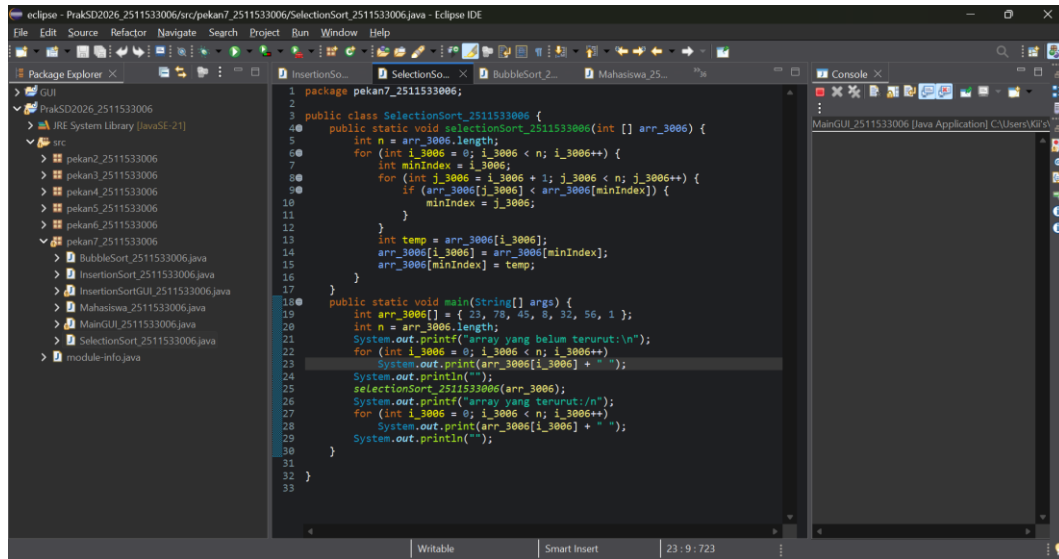


```
1 package pekan7_2511533006;
2
3 public class InsertionSort_2511533006 {
4     public static void InsertionSort_2511533006(int [] arr_3006) {
5         int n = arr_3006.length;
6         for (int i_3006 = 1; i_3006 < n ; i_3006++) {
7             int key = arr_3006 [i_3006];
8             int j_3006 = i_3006 - 1;
9             while (j_3006 >= 0 && arr_3006[j_3006] > key) {
10                 arr_3006[j_3006 + 1] = arr_3006[j_3006];
11                 j_3006--;
12             }
13             arr_3006[j_3006 + 1] = key; }
14         }
15
16     public static void main(String[] args ) {
17         int arr_3006[] = { 23, 78, 45, 8, 32, 56, 1 };
18         int n = arr_3006.length;
19         System.out.println("array yang belum terurut:\n");
20         for (int i_3006 = 0; i_3006 < n; i_3006++)
21             System.out.print(arr_3006[i_3006] + " ");
22         System.out.println("");
23         InsertionSort_2511533006(arr_3006);
24         System.out.println("array yang terurut:\n");
25         for (int i_3006 = 0; i_3006 < n; i_3006++)
26             System.out.print(arr_3006[i_3006] + " ");
27         System.out.println("");
28     }
29 }
30
31 }
32 }
```

Class InsertionSort_2511533006 mengimplementasikan algoritma yang membangun *array* terurut satu per satu dengan menyisipkan elemen ke posisi yang tepat.

Analisis: Program menelusuri *array* mulai dari indeks ke-1. Elemen pada iterasi saat ini disimpan dalam variabel *key*. Program kemudian membandingkan *key* dengan elemen-elemen sebelumnya (dari kanan ke kiri). Selama elemen sebelumnya lebih besar dari *key*, elemen tersebut digeser satu posisi ke kanan. Setelah menemukan posisi yang tepat, *key* disisipkan.

2.4 Program Selection Sort

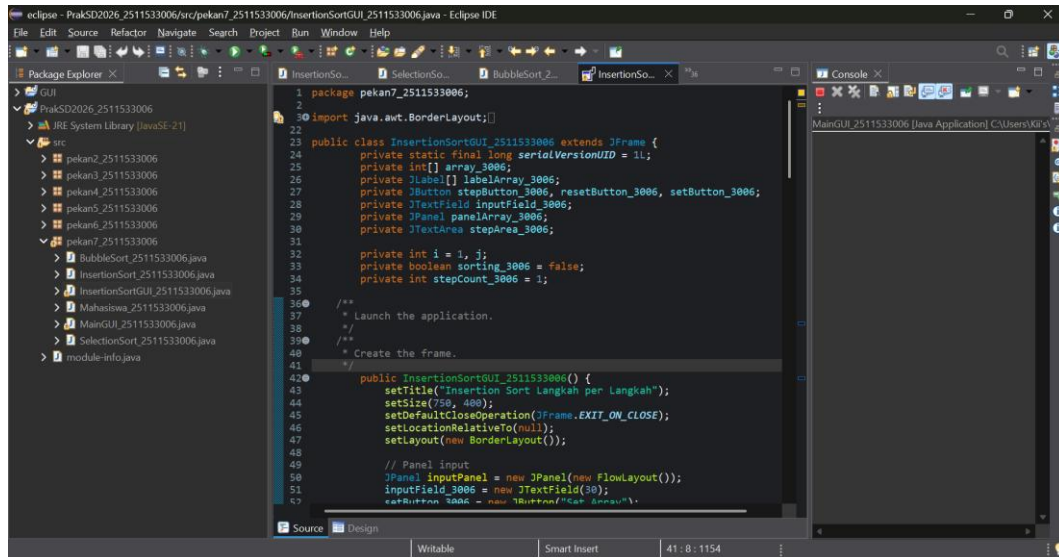


```
1 package pekan7_2511533006;
2
3 public class SelectionSort_2511533006 {
4     public static void selectionSort_2511533006(int [] arr_3006) {
5         int n = arr_3006.length;
6         for (int i_3006 = 0; i_3006 < n; i_3006++) {
7             int minIndex = i_3006;
8             for (int j_3006 = i_3006 + 1; j_3006 < n; j_3006++) {
9                 if (arr_3006[j_3006] < arr_3006[minIndex]) {
10                     minIndex = j_3006;
11                 }
12             }
13             int temp = arr_3006[i_3006];
14             arr_3006[i_3006] = arr_3006[minIndex];
15             arr_3006[minIndex] = temp;
16         }
17     }
18     public static void main(String[] args) {
19         int arr_3006[] = { 23, 76, 45, 8, 32, 56, 1 };
20         int n = arr_3006.length;
21         System.out.printf("array yang belum terurut:\n");
22         for (int i_3006 = 0; i_3006 < n; i_3006++) {
23             System.out.print(arr_3006[i_3006] + " ");
24         }
25         System.out.println("");
26         selectionSort_2511533006(arr_3006);
27         System.out.printf("array yang terurut:\n");
28         for (int i_3006 = 0; i_3006 < n; i_3006++) {
29             System.out.print(arr_3006[i_3006] + " ");
30         }
31         System.out.println("");
32     }
33 }
```

Class SelectionSort_2511533006 mengimplementasikan pengurutan dengan mencari nilai minimum dari bagian *array* yang belum terurut.

Analisis: Pada setiap iterasi perulangan luar, algoritma mencari nilai terkecil pada sisa elemen yang belum diurutkan (diperiksa melalui perulangan dalam). Indeks dari nilai terkecil tersebut disimpan dalam variabel *minIndex*. Setelah menelusuri seluruh sisa elemen, nilai pada *minIndex* ditukar dengan elemen pertama pada bagian yang belum terurut.

2.5 Program Insertion Sort GUI



Class InsertionSortGUI_2511533006 adalah implementasi berbasis Java Swing untuk memvisualisasikan cara kerja *Insertion Sort* langkah demi langkah.

Analisis: Program ini memungkinkan pengguna memasukkan *array* angka secara dinamis melalui `JTextField`. Setelah menekan "Set Array", angka-angka tersebut ditampilkan dalam bentuk kotak-kotak `JLabel`. Pengguna dapat menekan tombol "Langkah Selanjutnya" untuk melihat eksekusi algoritma *Insertion Sort* satu per satu. Setiap langkah yang terjadi akan dicatat dan di-*print* ke dalam komponen `JTextArea` (`stepArea_3006`), memudahkan pengguna memahami proses pergeseran dan penyisipan data pada *array*.

BAB III

KESIMPULAN

Algoritma pengurutan (*sorting*) merupakan fondasi penting dalam struktur data untuk menyusun informasi menjadi lebih terstruktur. Melalui praktikum ini, telah diimplementasikan tiga metode dasar: *Bubble Sort* yang melakukan pertukaran elemen berdekatan, *Selection Sort* yang menyeleksi nilai minimum, dan *Insertion Sort* yang menyisipkan elemen pada posisi yang sesuai. Meskipun ketiga algoritma ini memiliki kompleksitas waktu yang serupa pada kasus terburuk ($O(n^2)$), cara kerja logika masing-masing sangat berbeda.

Selain implementasi *console*, praktikum ini juga berhasil mengimplementasikan antarmuka grafis (GUI) untuk memvisualisasikan pergeseran indeks pada *Insertion Sort*. Pembuatan GUI ini membuktikan bahwa visualisasi log (*step-by-step*) sangat efektif dalam membantu pemahaman alur kerja suatu algoritma dibandingkan hanya melihat hasil akhir pengurutan.

DAFTAR PUSTAKA

- [1] H. M. Deitel dan P. J. Deitel, *Java: How to Program*, 11th ed. New Jersey, NJ: Pearson, 2017.

Tugas Struktur Data

1. Mahasiswa_2511533006

```
package pekan7_2511533006;

public class Mahasiswa_2511533006 {
    private String nama;
    private String nim;
    private String programStudi;

    public Mahasiswa_2511533006(String nama, String nim, String
programStudi) {
        this.nama = nama;
        this.nim = nim;
        this.programStudi = programStudi;
    }

    public String getNama_2511533006() { return nama; }
    public void setNama_2511533006(String nama) { this.nama =
nama; }

    public String getNim_2511533006() { return nim; }
    public void setNim_2511533006(String nim) { this.nim = nim; }

    public String getProgramStudi_2511533006() { return
programStudi; }
    public void setProgramStudi_2511533006(String programStudi)
{ this.programStudi = programStudi; }

    @Override
    public String toString() {
        return nama + " (" + nim + ") - " + programStudi;
    }
}
```

2. MainGUI_2511533006

```
3. package pekan7_2511533006;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;

public class MainGUI_2511533006 extends JFrame {
    private ArrayList<Mahasiswa_2511533006> listMahasiswa = new
ArrayList<>();

    private JTextField txtNama, txtNim, txtProdi;
    private JComboBox<String> comboSorting;
    private JTextArea txtVisualisasi;
    private JTable tableMahasiswa;
    private DefaultTableModel tableModel;
```

```

public MainGUI_2511533006() {
    setTitle("Sorting Data Mahasiswa - 2511533006");
    setSize(700, 600);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    // --- Panel Input ---
    JPanel panelInput = new JPanel(new GridLayout(4, 2, 5, 5));
    panelInput.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));

    panelInput.add(new JLabel("Nama Mahasiswa:"));
    txtNama = new JTextField();
    panelInput.add(txtNama);

    panelInput.add(new JLabel("NIM:"));
    txtNim = new JTextField();
    panelInput.add(txtNim);

    panelInput.add(new JLabel("Program Studi:"));
    txtProdi = new JTextField();
    panelInput.add(txtProdi);

    JButton btnTambah = new JButton("Tambah Data");
    JButton btnHapus = new JButton("Hapus Data Terakhir");
    panelInput.add(btnTambah);
    panelInput.add(btnHapus);

    // --- Panel Tabel (Data Mahasiswa) ---
    tableModel = new DefaultTableModel(new String[]{"Nama", "NIM", "Program Studi"}, 0);
    tableMahasiswa = new JTable(tableModel);
    JScrollPane scrollTable = new JScrollPane(tableMahasiswa);
    scrollTable.setPreferredSize(new Dimension(650, 150));

    // --- Panel Sorting & Visualisasi ---
    JPanel panelSorting = new JPanel(new BorderLayout());
    JPanel panelKontrolSort = new JPanel(new
    FlowLayout(FlowLayout.LEFT));

    String[] algos = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
    comboSorting = new JComboBox<>(algos);
    JButton btnMulaiSort = new JButton("Mulai Sorting");

    panelKontrolSort.add(new JLabel("Pilih Algoritma:"));
    panelKontrolSort.add(comboSorting);
    panelKontrolSort.add(btnMulaiSort);

    txtVisualisasi = new JTextArea();
    txtVisualisasi.setEditable(false);
    JScrollPane scrollVisual = new JScrollPane(txtVisualisasi);

    panelSorting.add(panelKontrolSort, BorderLayout.NORTH);
    panelSorting.add(scrollVisual, BorderLayout.CENTER);

    // --- Layout Utama ---

```

```

JPanel panelAtas = new JPanel(new BorderLayout());
panelAtas.add(panelInput, BorderLayout.NORTH);
panelAtas.add(scrollTable, BorderLayout.CENTER);

add(panelAtas, BorderLayout.NORTH);
add(panelSorting, BorderLayout.CENTER);

// --- Event Listeners ---
btnTambah.addActionListener(e -> tambahData_2511533006());
btnHapus.addActionListener(e -> hapusData_2511533006());
btnMulaiSort.addActionListener(e -> mulaiSorting_2511533006());
}

// Method tambah data mahasiswa via form GUI
public void tambahData_2511533006() {
    String nama = txtNama.getText().trim();
    String nim = txtNim.getText().trim();
    String prodi = txtProdi.getText().trim();

    if (!nama.isEmpty() && !nim.isEmpty() && !prodi.isEmpty()) {
        listMahasiswa.add(new Mahasiswa_2511533006(nama, nim, prodi));
        refreshTable_2511533006();
        txtNama.setText("");
        txtNim.setText("");
        txtProdi.setText("");
    } else {
        JOptionPane.showMessageDialog(this, "Semua field harus diisi!");
    }
}

// Method untuk menghapus data terakhir
public void hapusData_2511533006() {
    if (!listMahasiswa.isEmpty()) {
        listMahasiswa.remove(listMahasiswa.size() - 1);
        refreshTable_2511533006();
    }
}

// Method untuk memperbarui tampilan tabel GUI
public void refreshTable_2511533006() {
    tableModel.setRowCount(0);
    for (Mahasiswa_2511533006 mhs : listMahasiswa) {
        tableModel.addRow(new Object[]{mhs.getNama_2511533006(),
            mhs.getNim_2511533006(), mhs.getProgramStudi_2511533006()});
    }
}

// Method format visualisasi output
public String
getFormatNama_2511533006(ArrayList<Mahasiswa_2511533006> list) {
    StringBuilder sb = new StringBuilder("[");
    for (int i = 0; i < list.size(); i++) {
        sb.append(list.get(i).getNama_2511533006());
        if (i < list.size() - 1) sb.append(", ");
    }
    sb.append("]");
    return sb.toString();
}

```

```

}

// Method pemicu sorting berdasarkan ComboBox
public void mulaiSorting_2511533006() {
    if (listMahasiswa.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Data mahasiswa masih kosong!");
        return;
    }

    ArrayList<Mahasiswa_2511533006> dataSort = new
    ArrayList<>(listMahasiswa);
    String pilihan = (String) comboSorting.getSelectedItem();
    txtVisualisasi.setText("=== " + pilihan.toUpperCase() + " ===\n");
    System.out.println("=== " + pilihan.toUpperCase() + " ===");

    if (pilihan.equals("Insertion Sort")) {
        insertionSort_2511533006(dataSort);
    } else if (pilihan.equals("Selection Sort")) {
        selectionSort_2511533006(dataSort);
    } else if (pilihan.equals("Bubble Sort")) {
        bubbleSort_2511533006(dataSort);
    }

    listMahasiswa = dataSort;
    refreshTable_2511533006();
    txtVisualisasi.append("\nPengurutan Selesai!");
}

// --- IMPLEMENTASI ALGORITMA SORTING ---

// 1. Insertion Sort
public void insertionSort_2511533006(ArrayList<Mahasiswa_2511533006>
list) {
    for (int i = 1; i < list.size(); i++) {
        Mahasiswa_2511533006 key = list.get(i);
        int j = i - 1;

        while (j >= 0 &&
list.get(j).getNama_2511533006().compareToIgnoreCase(key.getNama_251
1533006()) > 0) {
            list.set(j + 1, list.get(j));
            j = j - 1;
        }
        list.set(j + 1, key);

        String step = "Langkah " + i + ": " +
getFormatNama_2511533006(list);
        txtVisualisasi.append(step + "\n");
        System.out.println(step);
    }
}

// 2. Selection Sort
public void selectionSort_2511533006(ArrayList<Mahasiswa_2511533006>
list) {
    for (int i = 0; i < list.size() - 1; i++) {
        int minIdx = i;

```

```

for (int j = i + 1; j < list.size(); j++) {
    if
(list.get(j).getNama_2511533006().compareToIgnoreCase(list.get(minIdx
x).getNama_2511533006()) < 0) {
        minIdx = j;
    }
}
Mahasiswa_2511533006 temp = list.get(minIdx);
list.set(minIdx, list.get(i));
list.set(i, temp);

String step = "Pass " + (i + 1) + ": " +
getFormatNama_2511533006(list);
txtVisualisasi.append(step + "\n");
System.out.println(step);
}
}

// 3. Bubble Sort
public void bubbleSort_2511533006(ArrayList<Mahasiswa_2511533006>
list) {
    int n = list.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (list.get(j).getNama_2511533006().compareToIgnoreCase(list.get(j
+ 1).getNama_2511533006()) > 0) {
                Mahasiswa_2511533006 temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
            }
        }
    }
    String step = "Pass " + (i + 1) + ": " +
getFormatNama_2511533006(list);
    txtVisualisasi.append(step + "\n");
    System.out.println(step);
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new MainGUI_2511533006().setVisible(true);
    });
}
}

```

4. Output

The screenshot displays the Eclipse IDE environment with a Java application titled "Sorting Data Mahasiswa - 2511533006". The application window includes input fields for "Nama Mahasiswa:", "NIM:", and "Program Studi:", along with "Tambah Data" and "Hapus Data Terakhir" buttons. A table lists student data:

Nama	NIM	Program Studi
Hafiz	2511531004	Informatika
Jose	2511538003	Informatika
Rizky	2511532006	Informatika
tirta	2511533606	Informatika
Zaki	2511533006	Informatika

The "Pilih Algoritma" dropdown is set to "Insertion Sort", and the "Mulai Sorting" button is visible. The console output shows the execution of the sorting algorithms:

```
==== INSERTION SORT ====
Langkah 1: [Rizky, Zaki, tirta, Hafiz, Jose]
Langkah 2: [Rizky, tirta, Zaki, Hafiz, Jose]
Langkah 3: [Hafiz, Rizky, tirta, Zaki, Jose]
Langkah 4: [Hafiz, Jose, Rizky, tirta, Zaki]
==== SELECTION SORT ====
Pass 1: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 2: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 3: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 4: [Hafiz, Jose, Rizky, tirta, Zaki]
==== BUBBLE SORT ====
Pass 1: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 2: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 3: [Hafiz, Jose, Rizky, tirta, Zaki]
Pass 4: [Hafiz, Jose, Rizky, tirta, Zaki]
==== INSERTION SORT ====
Langkah 1: [Hafiz, Jose, Rizky, tirta, Zaki]
Langkah 2: [Hafiz, Jose, Rizky, tirta, Zaki]
Langkah 3: [Hafiz, Jose, Rizky, tirta, Zaki]
Langkah 4: [Hafiz, Jose, Rizky, tirta, Zaki]
```